

AEP — ReUse+ — Parte 2

Implementação, POO e integração com MySQL

A Parte 2 implementa o núcleo funcional planejado na Parte 1 usando **Java 17**, Maven, JDBC e MySQL 8.x. A aplicação é executada no terminal e utiliza arquitetura em camadas, separando domínio, serviço, persistência e apresentação.

Funcionalidades implementadas

Funcionalidade	Implementação
Cadastro de doação	<code>DoacaoService.cadastrar()</code> chama <code>DoacaoRepository.criar()</code> e persiste em MySQL.
Consulta de doações	<code>DoacaoRepository.listar()</code> executa <code>SELECT</code> parametrizado e retorna uma lista de entidades.
Alteração de status	<code>DoacaoService.alterarStatus()</code> valida os estados aceitos antes do <code>UPDATE</code> .
Exclusão lógica	<code>DoacaoRepository.excluirLogicamente()</code> altera o status para <code>CANCELADA</code> , preservando o histórico.
POO	<code>Usuario</code> é abstrata; <code>Doador</code> e <code>Instituicao</code> herdam e sobreescrevem métodos com <code>@Override</code> .
Persistência	A conexão é feita por <code>DriverManager</code> e o driver oficial <code>mysql-connector-j</code> .

Estrutura de código

```
src/main/java/br/com/reuseplus/  
├── app/Main.java  
├── domain/  
│   ├── Usuario.java  
│   ├── Doador.java  
│   ├── Instituicao.java  
│   └── Doacao.java  
├── repository/DoacaoRepository.java  
└── service/DoacaoService.java
```

A classe `Usuario` define operações abstratas como `obterPermissoes()` e `descreverAtuacao()`. As subclasses implementam comportamentos diferentes, e o método `demonstrarPolimorfismo()` trabalha com um vetor do tipo `Usuario`, demonstrando despacho dinâmico. A entidade `Doacao` encapsula seus atributos e controla a alteração de status por método próprio.

Como executar

Primeiro, instale Java 17 ou superior, Maven e MySQL 8.x. Em seguida, execute o script `database/001_schema.sql` no MySQL. O script cria o banco `reuse_plus`, as tabelas, os índices e as categorias iniciais.

Configure as credenciais por variáveis de ambiente:

```
export REUSE_DB_URL='jdbc:mysql://localhost:3306/reuse_plus?useSSL=false&...'
export REUSE_DB_USER='root'
export REUSE_DB_PASSWORD='sua_senha'
```

Compile o projeto com:

```
mvn clean package
```

Execute a aplicação com:

```
mvn exec:java
```

Ou execute o JAR gerado com o driver disponível no classpath:

```
mvn dependency:build-classpath -Dmdep.outputFile=classpath.txt  
java -cp "target/reuse-plus-1.0.0.jar:$(cat classpath.txt)" br.com.reusep
```

Observação sobre dados iniciais

Para testar o cadastro de uma doação, é necessário inserir primeiro um usuário com `tipo_usuario = 'DOADOR'` e um item válido na tabela `item`. A aplicação usa os respectivos IDs no menu de terminal.

Critérios de aceite da Parte 2

A aplicação compila com Maven, utiliza uma entidade principal persistida em banco, executa operações de criação, consulta, alteração e exclusão lógica, usa classe abstrata, herança e polimorfismo com `@Override`, e mantém as regras de status na camada de serviço. A conexão real com o MySQL deve ser validada no ambiente da equipe com o banco em execução.

Limitações conhecidas e próximos incrementos

A versão acadêmica inicial concentra o CRUD de doações para garantir um núcleo funcional. O próximo incremento recomendado é implementar CRUD de usuários, categorias e itens, seguido pelo módulo de solicitações, aprovação, entrega e histórico. Esses incrementos devem preservar os requisitos e o DER definidos na Parte 1.